Qualidade de testes
Detecção e refatoração
de Test Smells

Disciplina: Teste de Software (conforme README)

Nome completo: Não informado

Matrícula: Não informada

Projeto: UserService - laboratório de testes com Jest

Repositório: https://github.com/MateusRodrigu/test-smelly

Data: 18 de outubro de 2026

1. Análise manual dos smells

A suíte original está em **test/userService.smelly.test.js**. A execução inicial aprovou quatro testes e ignorou um. Esse resultado não garante a qualidade das verificações: há caminhos que podem passar sem avaliar o comportamento esperado.

Eager Test: responsabilidades misturadas

O primeiro teste cria e busca um usuário, alternando ações e asserções. Uma falha na criação impede avaliar a busca e torna o diagnóstico menos preciso. A refatoração dedica um teste à criação e outro à consulta, usando a criação apenas como preparação neste último.

Conditional Test Logic: fluxo dentro do teste

O teste de desativação usa for e if/else para tratar usuário comum e administrador. As verificações dependem do caminho executado, dificultando perceber cenários omitidos. Dois testes independentes tornam explícitos o sucesso da desativação e a proteção do administrador.

Teste frágil: acoplamento ao layout

O teste de relatório exige uma linha com pontuação, ordem de campos, quebra de linha e cabeçalho exatos. Mudanças cosméticas podem provocar falhas sem regressão funcional. A nova suíte verifica nomes, IDs e estados relevantes sem exigir a apresentação completa. Essa escolha pressupõe que o layout exato não é um contrato público, coerente com o comentário do serviço.

Outras observações

No teste de idade, o expect dentro de catch pode nunca executar: se a validação desaparecer, o teste passa. O novo toThrow exige o erro. O test.skip deixa o cenário vazio sem cobertura; a nova suíte verifica a mensagem do relatório vazio, pois não existe método público de listagem.

O banco é interno e em memória, sem evidência de Convidado Misterioso externo. Como ele é compartilhado pelo módulo, cada teste usa _clearDB() no setup. Esse acoplamento fica restrito ao mecanismo de isolamento fornecido pelo projeto.

2. Processo de refatoração

O cenário de desativação foi separado em dois contratos. Abaixo está o teste original completo e os dois testes que o substituem.

Antes (comentários omitidos)

```
test('deve desativar usuários se eles não forem administradores', () => {
    const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
    const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

    const todosOsUsuarios = [usuarioComum, usuarioAdmin];

    for (const user of todosOsUsuarios) {
      const resultado = userService.deactivateUser(user.id);
      if (!user.isAdmin) {
        expect(resultado).toBe(true);
        const usuarioAtualizado = userService.getUserById(user.id);
        expect(usuarioAtualizado.status).toBe('inativo');
      } else {
        expect(resultado).toBe(false);
      }
    }
  });
```

Depois

```
test('desativa um usuário comum e mantém seu cadastro acessível', () => {
    const usuario = userService.createUser('Comum', 'comum@teste.com', 30);

    const resultado = userService.deactivateUser(usuario.id);

    expect(resultado).toBe(true);
    expect(userService.getUserById(usuario.id)).toMatchObject({
      id: usuario.id, status: 'inativo',
    });
  });

  test('preserva o administrador ativo ao recusar sua desativação', () => {
    const admin = userService.createUser('Admin', 'admin@teste.com', 40, true);

    const resultado = userService.deactivateUser(admin.id);

    expect(resultado).toBe(false);
    expect(userService.getUserById(admin.id)).toMatchObject({
      id: admin.id, status: 'ativo', isAdmin: true,
    });
  });
```

Cada bloco prepara um usuário, executa uma ação e verifica o resultado e o estado observável (AAA), separados por linhas em branco. Não há decisões nem repetição no fluxo do teste. A verificação do administrador também exige que ele permaneça ativo, cobrindo uma lacuna do original.

3. Detecção automática e validação

Foi usado ESLint 8.57.1 para executar diretamente o formato legado .eslintrc.json pedido no roteiro. A configuração habilita as regras recomendadas e as três regras explícitas. O lockfile registra as versões instaladas.

```
$ npx eslint .
.\test\userService.smelly.test.js
  44:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  46:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  49:9  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  73:7  error    Avoid calling `expect` conditionally`  jest/no-conditional-expect
  77:3  warning  Tests should not be skipped            jest/no-disabled-tests
  77:3  warning  Test has no assertions                 jest/expect-expect

X 6 problems (4 errors, 2 warnings)
```

Figura 1. Reprodução visual da saída real da primeira execução; o registro textual acompanha o projeto. Caminho absoluto abreviado e símbolo de erro normalizado para legibilidade.

As quatro asserções condicionais são erros; o teste ignorado e a ausência de asserções nele geram avisos. A regra de títulos idênticos não encontra duplicatas. A ferramenta detecta padrões sintáticos, mas não identifica automaticamente o Eager Test nem decide se uma comparação de relatório é frágil.

Validação final: 13 testes aprovados e 1 ignorado, em 2 suítes aprovadas. A suíte limpa passa no ESLint com --max-warnings 0. O lint global mantém os 4 erros e 2 avisos do original preservado. O registro completo está em docs/testes-finais.txt. O npm também reportou 33 vulnerabilidades nas dependências; sua atualização exige avaliação separada de compatibilidade.

4. Conclusão

Testes pequenos e diretos facilitam localizar regressões. Asserções incondicionais evitam sucesso sem verificação, enquanto o isolamento reduz dependências entre casos. Análise estática fornece feedback repetível e complementa a revisão humana de intenção, cobertura e acoplamento. Passar no linter não basta para provar a qualidade de uma suíte.

Referências:

github.com/jest-community/eslint-plugin-jest
eslint.org/docs/latest/use/configure/migration-guide